Travaux Dirigés N°5 OS202

Jair VASQUEZ TORRES

Introduction

Le calcul scientifique moderne se heurte de plus en plus aux limites physiques des processeurs traditionnels (CPU). Bien que les CPU soient optimisés pour des tâches séquentielles complexes et la gestion de logique de contrôle (grâce à des caches larges et des unités de prédiction de branchement), ils ne possèdent qu'un nombre limité de cœurs. À l'inverse, l'architecture **GPU** (Graphics Processing Unit) est conçue pour le calcul massivement parallèle.

L'importance du passage au GPU réside dans le débit de données (*throughput*). Là où un CPU traite les instructions les unes après les autres, le GPU peut exécuter des milliers de threads simultanément. Cette approche est particulièrement efficace pour les problèmes dits "parfaitement parallèles", tels que le traitement d'images ou les simulations fractales, où le calcul d'un élément ne dépend pas de ses voisins [1]. L'objectif de ce TP est de maîtriser l'interface **PyCUDA** pour exploiter cette puissance, en passant d'une exécution scalaire sur l'hôte (Host) à une exécution vectorisée sur le périphérique (Device).

1 Environnement de Test

Le développement et les mesures de performance ont été effectués sur la plateforme cloud **Google Colab**. Ce choix est stratégique car il permet d'accéder à du matériel de classe centre de données sans infrastructure locale complexe.

- **GPU :** NVIDIA Tesla T4 (Architecture Turing).
- **Cœurs CUDA :** 2560 cœurs, optimisés pour la précision simple (FP32).
- **Interface :** PyCUDA 2024.1, servant de pont entre Python et les noyaux C++.

L'utilisation de la Tesla T4 permet de bénéficier de la technologie de mémoire **GDDR6**, cruciale pour minimiser les goulots d'étranglement lors du transfert de données entre le Host et le Device via le bus PCIe.

2 Somme de deux vecteurs

La somme de deux vecteurs $\vec{C} = \vec{A} + \vec{B}$ est l'opération la plus simple pour illustrer le parallélisme. Chaque élément i peut être calculé indépendamment : $C[i] = A[i] + B[i]$.

2.1 Organisation des threads et blocs

Dans CUDA, les threads sont regroupés en blocs. Pour cet exercice, nous avons utilisé une taille de bloc de **256 threads**. Ce nombre n'est pas arbitraire :

- **Le Warp :** Les GPU NVIDIA exécutent les threads par groupes de 32 (appelés *warps*). Une taille de bloc multiple de 32 (comme 256) garantit qu'aucune unité de calcul ne reste oisive.
- **Occupation :** 256 est un équilibre optimal qui permet au planificateur (*scheduler*) de la Tesla T4 de maintenir une haute occupation des multiprocesseurs de flux (SM).

L'index global de chaque élément est calculé dans le kernel via : `int i = threadIdx.x + blockIdx.x * blockDim.x;`

3 Somme de deux matrices carrées

L'opération $C_{i,j} = A_{i,j} + B_{i,j}$ illustre la flexibilité de CUDA pour traiter des structures multidimensionnelles. Bien que la mémoire physique du GPU soit strictement linéaire, nous avons implémenté deux approches pour interpréter ces données.

3.1 L'illusion de la structure : Mémoire Linéaire

Pour le GPU, une matrice $N \times N$ n'existe pas intrinsèquement. Le CPU envoie un pointeur vers un segment de mémoire contigu contenant N^2 éléments. La distinction entre l'approche "Vector" et "2D" ne réside pas dans le stockage des données, mais dans l'organisation des threads pour y accéder.

3.2 Approche 1 : Linéarisation (Comment Vector)

Cette méthode traite la matrice comme un tableau unidimensionnel géant.

- **Configuration :** Utilisation d'une grille 1D avec `block=(256,1,1)`.
- **Indexation :** Chaque thread calcule un index unique $idx = \text{threadIdx.x} + \text{blockIdx.x} \times \text{blockDim.x}$.
- **Sécurité :** La condition `if (idx < dim*dim)` est cruciale pour éviter les accès hors limites si la taille de la grille n'est pas un multiple exact de 256.

3.3 Approche 2 : Topologie 2D (Comment 2D)

Ici, nous forçons une correspondance entre la topologie de la grille de calcul et la forme logique de la matrice.

- **Configuration :** Blocs 2D de `(16,16,1)`. Chaque thread calcule ses coordonnées (x, y).

— **Calcul de l'index :** Pour retrouver l'adresse dans la mémoire linéaire, le kernel effectue la conversion manuelle : `idx = x + (y * dim)`.

3.4 Gestion du typage et sécurité mémoire

L'implémentation a nécessité une gestion rigoureuse des types de données pour correspondre à l'architecture C++ du GPU :

— **Précision simple :** Les matrices ont été converties en `float32`. Les cœurs CUDA de la Tesla T4 sont optimisés pour ce format, offrant des performances nettement supérieures au `float64`.

— **Typage strict :** Les dimensions sont passées via `np.int32(dim)`. Sans cette conversion, le kernel recevrait un objet Python complexe au lieu d'une valeur entière de 4 octets, provoquant une erreur de lecture dans les registres du GPU.

— **Protection des bords :** Puisque notre matrice (20×20) n'est pas un multiple exact de la taille du bloc (16×16), la grille générée est de 32×32. La condition `if (x < dim && y < dim)` est donc indispensable pour que les threads "excédentaires" ne corrompent pas la mémoire.

Paramètre	**App. Vector**	**App. 2D**
Configuration Bloc	$(256, 1, 1)$	$(16, 16, 1)$
Nombre de Blocs	$\lceil N^2/256 \rceil$	$(\lceil N/16 \rceil, \lceil N/16 \rceil)$
Type de données	`float32`	`float32`
Vérification limites	`idx<dim*dim`	`x<dim && y<dim`

Table 1 – Synthèse des paramètres de configuration pour les deux méthodes de somme matricielle.

4 Mandelbrot : Analyse de Performance et Accélération

Le calcul du fractal de Mandelbrot est l'exemple le plus probant de la puissance du calcul parallèle. Chaque point du plan complexe étant calculé de manière totalement indépendante, nous pouvons exploiter pleinement l'architecture massive de la Tesla T4.

4.1 Comparaison des résultats

En utilisant une grille de 1000×1000 divisions et 100 itérations, nous avons obtenu les mesures de temps suivantes :

Dispositif	**T. d'exéc. (s)**	**Accélération**
CPU (Numpy)	0.9066 s	$1\times$ (Référence)
GPU (PyCUDA)	**0.0003 s**	**3022**$\times$

Table 2 – Performances mesurées pour le calcul de Mandelbrot.

4.2 Analyse de l'accélération massive

L'accélération de plus de **3000x** observée s'explique par la combinaison de plusieurs facteurs critiques :

— **Élimination des calculs inutiles :** Dans le noyau CUDA, nous avons implémenté un `break` immédiat dès que $|z| > 2$. Dans la version CPU (Numpy), même si un point diverge, le processeur continue souvent de calculer des opérations sur des matrices complètes, gaspillant des cycles.

— **Latence de mémoire :** Le CPU doit aller chercher les données dans la RAM principale. Dans notre noyau GPU, les variables `z_re` et `z_im` sont stockées dans des **registres**, l'unité de mémoire la plus rapide du matériel, accessible en un seul cycle d'horloge.

— **Parallélisme au niveau des pixels :** Avec 1 000 000 de points à calculer, la Tesla T4 répartit cette charge sur ses milliers de cœurs CUDA, traitant des blocs entiers de l'image simultanément là où le CPU traite les données de façon séquentielle ou par petits vecteurs SIMD.

5 Conclusion

Ce TP nous a permis de valider l'efficacité de l'architecture **CUDA** pour le calcul scientifique intensif. Les conclusions principales sont les suivantes :

1. **Supériorité du GPU sur les calculs indépendants :** Pour des tâches simples comme la somme de vecteurs ou de matrices, le gain est notable, mais c'est sur des calculs itératifs complexes (Mandelbrot) que la GPU montre sa véritable supériorité, atteignant des facteurs d'accélération impossibles à obtenir avec une optimisation CPU classique.
2. **Importance de l'indexation :** Nous avons vu que la structure 2D (`x, y`) n'est qu'une abstraction logique pour le programmeur. La maîtrise de la **linéarisation de l'index** ($idx = y \times div + x$) est essentielle pour communiquer correctement avec la mémoire physique du GPU.
3. **Impact sur le flux de travail :** Réduire un temps de calcul de presque une seconde à moins d'un tiers de milliseconde change radicalement la portée des simulations possibles, permettant d'augmenter la résolution (plus de divisions) ou la précision (plus de boucles) sans pénalité de temps sensible.

L'utilisation de la bibliothèque **PyCUDA** s'est avérée être un outil puissant, combinant la flexibilité de Python pour le post-traitement (Génération des graphiques avec Matplotlib) et la performance brute du C++ pour les calculs intensifs sur le "Device".

Ressources et Reproductibilité

L'intégralité du code et du Makefile est accessible sur : `https://github.com/JairVasquezT/OS202-2026.git`

Références

[1] NVIDIA Corporation, *CUDA C++ Programming Guide*, v12.1, 2023.